

Data Cleaning & QA Documentation

Healthcare Claims Automation — Practice Dataset

1. Purpose

This document records the data extraction, profiling, transformation, validation, and quality-assurance activities performed on the healthcare claims practice dataset.

The objective is to demonstrate a reusable healthcare ETL workflow in Python and document how source data was standardized and how data-quality issues were identified, handled, and reported.

The process separates three major responsibilities:

- **Transformation:** Convert source data into a consistent, usable structure.
- **Validation:** Checking whether the transformed data satisfies expected data-quality and business rules.
- **QA Documentation:** Recording whether an identified issue was corrected, removed, flagged for review, or retained.

2. Dataset Used

2.1 Monthly Claims Data

Three monthly healthcare claims files were used.

Source File	Rows
claims_january_2026.csv	46
claims_february_2026.csv	40
claims_march_2026.csv	43
Total source rows	129

The three monthly files were standardized and combined.

Two exact duplicate rows were removed during transformation, resulting in:

127 rows in the transformed claims dataset.

A separate conflicting duplicate **claim_id** was identified during validation. Because the two records contained different claim-level information, they were retained and flagged rather than automatically removed.

2.2 Reference Data

Reference data was used to validate member and provider identifiers.

Reference File	Records
providers.csv	12 providers
members.xlsx	30 members across 2 sheets

The two member sheets were combined into a single reference dataset. A **member_type** column was added to preserve the original sheet classification.

3. ETL Process

The pipeline follows this structure:

1. **Extraction**
2. **Data Profiling**
3. **Transformation / Cleaning**
4. **Validation**
5. **QA Flagging**
6. **Load / Output**

4. Extraction

The monthly CSV claims files were extracted using pandas.

For each source file:

- The CSV file was loaded into a pandas DataFrame.
- The original filename was stored in the **source_file** column.
- A **load_timestamp** was added to record when the data entered the pipeline.

Reference data was also extracted:

- **providers.csv** was loaded as provider reference data.
- Both sheets of **members.xlsx** were loaded.
- The member sheets were combined into one DataFrame.
- **member_type** was added to preserve the original member-sheet classification.

The **source_file** field provides source-level lineage, allowing each claim to be traced back to its original monthly file.

5. Data Profiling

Data profiling was performed before transformation to understand the structure and characteristics of each source dataset.

The profiling process examined:

- Number of rows and columns
- Column names
- Data types

- Missing values
- Duplicate rows
- Sample records
- Unique values
- Numeric distributions

Important source differences identified

The profiling stage identified several differences across the monthly source files:

- Different column names across monthly files
- Different representations of **service_date**
- **procedure_code** was interpreted as an integer in the source data
- Inconsistent casing and whitespace in **claim_status**
- Some columns were missing from particular source files
- Missing values were present in several columns
- Exact duplicate rows were present
- Duplicate **claim_id** values were present
- Negative values were present in **billed_amount**
- Some records contained **paid_amount** greater than **billed_amount**

These findings were used to define the transformation and validation steps.

6. Transformation

6.1 Column Standardization

The monthly files used different column names for some of the same business fields. These source-specific names were mapped to a common target schema.

Target Column	January	February	March	Target Data Type
claim_id	claim_id	ClaimID	claim_id	String
member_id	member_id	MemberID	member_id	String
provider_id	provider_id	ProviderID	rendering_provider	String
service_date	service_date	ServiceDate	service_date	Date
diagnosis_code	diagnosis_code	DiagnosisCode	diag_code	String
procedure_code	procedure_code	ProcedureCode	proc_code	String
claim_type	claim_type	—	claim_type	String
billed_amount	billed_amount	BilledAmount	billed_amt	Decimal
paid_amount	paid_amount	PaidAmount	paid_amt	Decimal
claim_status	claim_status	ClaimStatus	status	String
denial_reason	—	—	denial_reason	String
source_file	source_file	source_file	source_file	String
load_timestamp	load_timestamp	load_timestamp	load_timestamp	Datetime

Missing target columns were added to the transformed dataset where they were not available in the source file.

6.2 Service Date Standardization

The source files represented **service_date** using different date formats.

Source File	Source Format
claims_january_2026.csv	YYYY-MM-DD
claims_february_2026.csv	MM/DD/YYYY
claims_march_2026.csv	YYYY-MM-DD

Source-specific date formats were used when parsing the files. The resulting values were converted into pandas datetime representation. Invalid date values were converted to NaT using `errors="coerce"`

This allows invalid dates to be identified later during validation rather than causing the entire transformation process to fail.

6.3 Procedure Code Data-Type Standardization

`procedure_code` was interpreted as an integer in some source data.

It was converted to a string because procedure codes represent identifiers rather than numeric measures.

This allows the pipeline to validate the code based on its format rather than treating it as a value on which mathematical operations should be performed.

6.4 Claim Status Standardization

The source data contained differences in casing and whitespace for `claim_status`.

The status field was standardized by:

- Removing leading and trailing whitespace
- Applying consistent title-case formatting

Example standardized values include:

Standardized Claim Status
Denied
Paid
Pending
Approved

This standardization is important because the validation rules depend on consistent status values. For example, the rule for a denied claim with a missing denial reason checks for the standardized value **Denied**.

6.5 Exact Duplicate Removal

Two exact duplicate rows were identified across the monthly source data.

Because the duplicated rows contained identical values across the complete record, they were removed during transformation.

After duplicate removal:

129 source rows —> 127 transformed rows

The transformed dataset contained no remaining exact duplicate rows.

7. Data Quality Validation

Validation was performed after transformation and standardization.

The validation rules were designed as row-level Boolean checks so that individual claims could be flagged with specific QA reasons.

7.1 Missing Required Claim ID

claim_id was treated as the key identifier for the claim records.

Validation result:

Check	Failed Rows
Missing <code>claim_id</code>	0

No claims were missing `claim_id`.

7.2 Missing Values

Missing values were reviewed by column rather than treating every missing value as a data-quality failure.

This distinction is important because some fields are not applicable to every claim.

member_id	3	Flagged and retained
service_date	3	Flagged and retained
claim_type	40	Missing in source files where the field was unavailable; not treated as a blanket failure
denial_reason	122	Conditionally validated

Denial Reason

denial_reason is only expected when a claim has a denied status.

Therefore, the pipeline does not treat every missing **denial_reason** as an error.

Instead, the validation rule checks:

If **claim_status = Denied**, then **denial_reason** should not be missing.

Validation result:

- 122 total missing **denial_reason** values
- 37 denied claims were missing a denial reason and were flagged
- The remaining missing values were associated with claims where a denial reason was not required by this rule

8. Duplicate Validation

8.1 Exact Duplicate Rows

Two exact duplicate rows were found during transformation.

They were removed because the complete records were identical.

After transformation:

Exact duplicate rows remaining: 0

8.2 Duplicate Claim ID

A separate duplicate claim_id was identified during validation.

The same claim_id appeared in two records with different:

- member_id
- provider_id
- service_date
- diagnosis_code
- procedure_code
- Other claim-level values

Because these records contained conflicting information, the pipeline did **not** automatically select one record or delete both records.

Instead:

The conflicting duplicate claim_id was retained and flagged for review.

Validation result:

Check	Failed Rows
Duplicate claim_id	2

The count is 2 because both rows associated with the duplicated claim ID are flagged.

9. Data Type and Format Validation

9.1 Procedure Code

Procedure codes were converted from integer representation to string representation during transformation.

They were then checked against the expected five-digit procedure-code format.

Validation pattern:

`r'^\d{5}$'`

This pattern requires:

- Exactly five characters
- All characters must be digits

Validation result:

Check	Failed Rows
Invalid procedure code	0

No invalid procedure codes were identified.

9.2 Diagnosis Code

Diagnosis codes were checked using the following validation pattern:

`r'^[A-Z0-9]{3}(\.[0-9]{1,3})?$'`

The pattern allows:

- Three alphanumeric characters

or

- Three alphanumeric characters followed by a decimal point and one to three digits

Validation result:

Check	Failed Rows
Invalid diagnosis code	0

No invalid diagnosis codes were identified.

10. Claim Status Consistency

After trimming whitespace and standardizing casing, the claim status values were normalized.

The resulting categories were:

Claim Status	Rows
Denied	42
Paid	36
Pending	33
Approved	16
Total	127

The standardized status values were then used by downstream validation rules, including the conditional **denial_reason** check.

11. Numeric Business Rule Validation

Numeric validation was performed on **billed_amount** and **paid_amount**.

11.1 Negative Billed Amount

A negative billed amount was treated as a data-quality issue and flagged.

Rule	Failed Rows
Negative billed_amount	3

Validation rule:

```
combined_claims['billed_amount'] < 0
```

The affected records were retained in the dataset and marked as QA failures rather than automatically deleted.

11.2 Paid Amount Greater Than Billed Amount

The pipeline also checked whether the amount paid was greater than the amount billed.

Rule	Failed Rows
paid_amount > billed_amount	3

Validation rule:

```
combined_claims['paid_amount'] > combined_claims['billed_amount']
```

These records were flagged for review.

The pipeline does not automatically modify the monetary values because the source records should be reviewed before deciding whether the values represent source-data errors or a legitimate business scenario.

12. Referential Integrity Validation

Reference data was used to check whether claim-level member and provider identifiers existed in the corresponding reference datasets.

12.1 Provider Validation

The claim **provider_id** values were compared against the valid provider IDs from **providers.csv**.

Check	Invalid Rows
Invalid provider ID	0

No invalid provider IDs were identified.

12.2 Member Validation

The claim **member_id** values were compared against the valid member IDs from **members.xlsx**.

Check	Invalid Rows
Invalid member ID	0

No invalid member IDs were identified.

13. QA Validation Summary

The final validation results were:

Validation Rule	Failed Rows
Missing claim_id	0
Exact duplicate rows remaining	0
Duplicate claim_id	2
Invalid procedure code	0
Invalid diagnosis code	0
Invalid provider ID	0
Invalid member ID	0
paid_amount > billed_amount	3
Negative paid_amount	0
Negative billed_amount	3
Denied claim with paid amount > 0	0
Denied claim missing denial_reason	37

Important interpretation

The individual validation-rule counts should not be added together to determine the number of failed claims.

A single claim can fail multiple validation rules.

For example, one claim could simultaneously have:

- a negative billed_amount
- paid_amount > billed_amount

- and a duplicate claim_id

Therefore, the pipeline also creates an overall qa_status and a qa_reason field for each claim.

14. QA Flagging

Each transformed claim receives a QA status:

- pass
- fail

When a validation rule fails, the corresponding validation name is added to the qa_reason field.

For example:

qa_status = fail

qa_reason = duplicate_claim_id;negative_billed_amount;

This approach preserves the original record while clearly documenting why the record failed validation.

The current pipeline produced:

QA Status	Rows
Pass	88
Fail	39
Total	127

The 39 failed rows represent **unique claim rows with at least one QA failure**. This number is lower than the sum of individual rule failures because some rows fail more than one rule.

15. Data Quality Issue Handling

The pipeline uses different handling strategies depending on the type of issue.

Issue Type	Action
Exact duplicate rows	Removed during transformation
Conflicting duplicate claim_id	Retained and flagged
Missing claim_id	Flagged
Missing member_id	Flagged and retained
Missing service_date	Flagged and retained
Missing claim_type	Reviewed as source/schema difference
Missing denial_reason on denied claims	Flagged
Invalid procedure code	Flagged if detected
Invalid diagnosis code	Flagged if detected
Negative billed amount	Flagged
Paid amount greater than billed amount	Flagged
Invalid provider ID	Flagged if detected
Invalid member ID	Flagged if detected

The pipeline intentionally avoids silently changing questionable business data. Where the correct value cannot be determined from the available source data, the record is retained and flagged for review.

16. Output

The pipeline produces two main output datasets:

Processed Claims

processed_claims.csv

Contains the transformed claims dataset with QA status and QA reason information.

QA Failed Claims

qa_failed_claims.csv

Contains the records that failed at least one validation rule.

The final processed dataset contains:

127 claims

with:

- **88 QA-passing records**
- **39 QA-failing records**

17. Conclusion

This project demonstrates a reusable healthcare claims ETL and data-quality workflow using Python and pandas.

The workflow covers:

1. Extracting multiple source files
2. Preserving source lineage
3. Profiling source data
4. Standardizing different source schemas
5. Standardizing data types and formats
6. Removing exact duplicate records
7. Validating business and data-quality rules
8. Checking referential integrity against reference data
9. Applying row-level QA flags
10. Producing processed and QA-failure outputs

A key design principle of the pipeline is that **not every missing or unusual value is automatically treated as an error.**

Instead, validation rules are based on the expected meaning of each field. For example, `denial_reason` is conditionally required for denied claims, while missing values in fields that are not available from a particular source are documented separately.

This allows the pipeline to distinguish between:

- issues that can be safely corrected,
- duplicate records that can be removed,
- records that require investigation,
- source-specific missing fields,
- and values that are validly missing based on business context.